By default, these classes will be direct descendents of Object. You can give them a different parent by passing the parent's class as a parameter:

```
some_class = Class.new(String) do
  def vowel_movement
    tr 'aeiou', '*'
  end
end

obj = some_class.new("now is the time")
puts obj.vowel_movement
```

produces:

```
n*w *s th* t*m*
```

How Classes Get Their Names

You may have noticed that the classes created by Class.new have no name. However, if you assign the class object for a class with no name to a constant, Ruby automatically names the class after the constant:

```
some_class = Class.new
obj = some_class.new
puts "Initial name is #{some_class.name}"
SomeClass = some_class
puts "Then the name is #{some_class.name}"
puts "also works via the object: #{obj.class.name}"
```

produces:

```
Initial name is
Then the name is SomeClass
also works via the object: SomeClass
```

We can use these dynamically constructed classes to extend Ruby in interesting ways. For example, here's a simple reimplementation of the Ruby Struct class:

```
def MyStruct(*keys)
  Class.new do
    attr_accessor *keys

    def initialize(hash)
      hash.each do |key, value|
        instance_variable_set("@#{key}", value)
      end
    end
  end
end

Person = MyStruct :name, :address, :likes
dave = Person.new(name: "dave", address: "TX", likes: "Stilton")
chad = Person.new(name: "chad", likes: "Jazz")
chad.address = "Berlin"

puts "Dave's name is #{dave.name}"
puts "Chad lives in #{chad.address}"
```

produces:

```
Dave's name is dave
Chad lives in Berlin
```